

UNIDADE 3

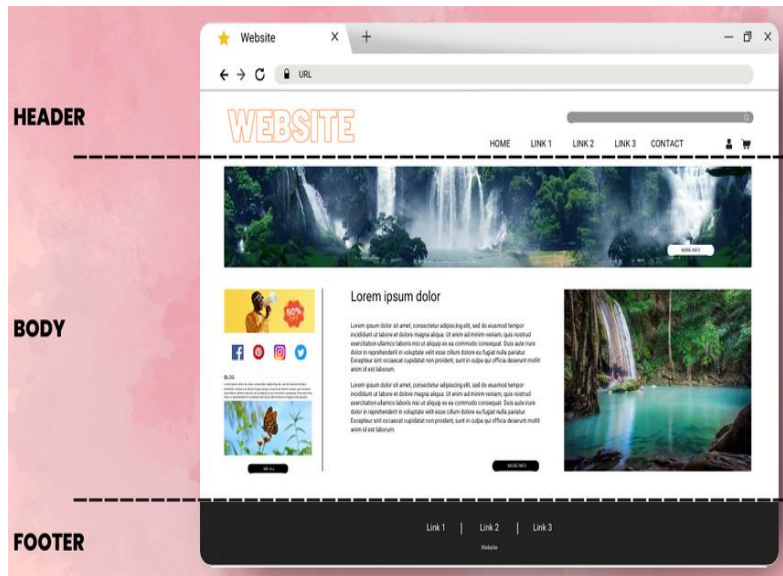
Navegación e Comunicación entre Componentes

Obxectivos da unidade

- Integración de **elementos fixos** como **navbar e footer** en múltiples vistas
- Comprender e aplicar o **enrutamento** en Vue para crear múltiples vistas dentro dunha SPA, rutas básicas, navegación entre compoñentes y parámetros na URL
- Aprender a comunicar compoñentes entre si mediante **props** e eventos personalizados, envío de eventos mediante **\$emit** e introdución a os *slots*
- Aplicar novos estilos dinámicos usando **v-bind:class** e **v-bind:style**

A.1. Compoñentes fixos

Antes de mergullarnos de cheo no enrutamento, é interesante comezar definindo dous compoñentes que van aparecer en todas as páxinas que o encabezamento ou **Header** e o Pé ou **Footer**. Para nós serán os compoñentes *NavBar.vue* e o *Footer.vue* situado no almacén desta unidade.



Estes elementos adoitan ser *compoñentes fixos*, xa que non cambian ao navegar entre rutas. Grazas a eles temos **unha estrutura común** en toda a aplicación: un menú na parte superior que permita moverse entre vistas, e un pé de páxina que complete o deseño.

Descargamos estes compoñentes e os engadimos na carpeta **components** e facemos as **modificacións** que se sinalan a continuación en **App.vue**.

```

File Edit Selection View Go Run Terminal Help
XestionUsuarios.vue M NavBar.vue U Footer.vue U App.vue M X
EXPLORER
PROXECTOVUE
  .vscode
  node_modules
  public
  src
    assets
    components
      Footer.vue U
      NavBar.vue U
      XestionUsuarios.vue M
      App.vue M
    main.js
    style.css
  .gitignore
  index.html
  package-lock.json
  package.json
  README.md
  vite.config.js
src > App.vue > {} style > global(body)
1 <template>
2   <div id="app">
3     <NavBar />
4     <!-- Aquí se carga el componente principal DE MOMENTO-->
5     <XestionUsuarios />
6     <Footer />
7   </div>
8 </template>
9
10 <script setup>
11 import NavBar from './components/NavBar.vue'
12 import Footer from './components/Footer.vue'
13 import XestionUsuarios from './components/XestionUsuarios.vue'
14 </script>
15
16 <style>
17 /* Estilos globales mínimos */
18 :global(body) {
19   margin: 0;
20   background-color: #f6f6f6;
21 }
22
23 #app {
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

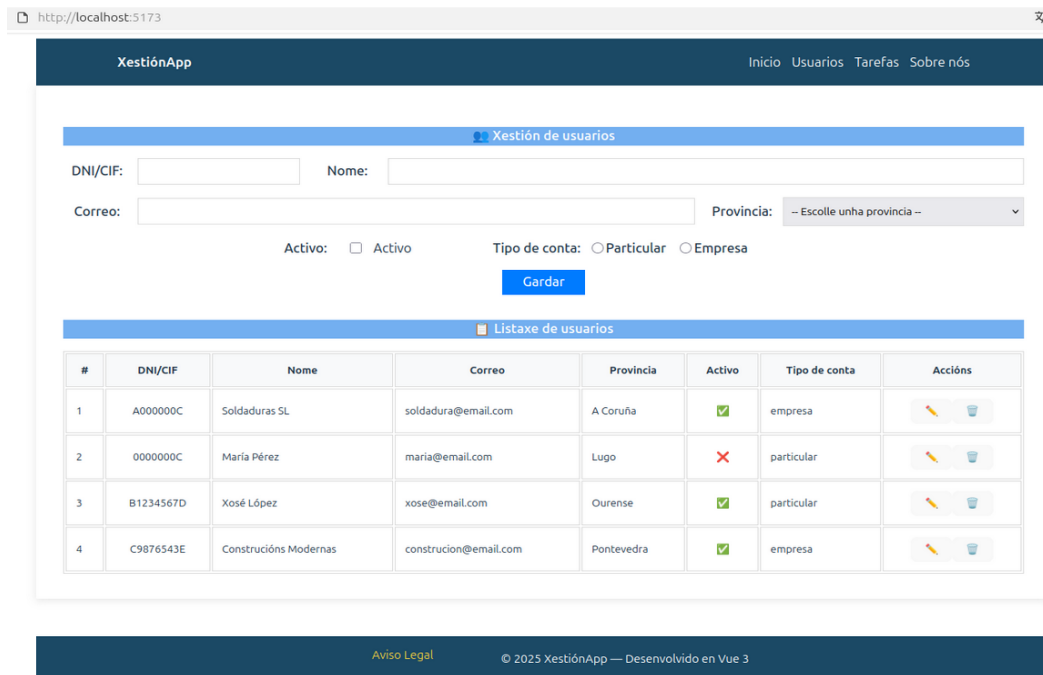
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
usuario@cursovvue:~/proxectovue$ npm run dev

VITE v7.1.12 ready in 439 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

```

O resultado final sería xa se aprecia o *NavBar* ou navegador da aplicación e un típico *Footer* sinxelo.



O código máis sinxelo é o de *Footer.vue*, onde pode aparecer aspectos como avisos legais, contactos, acerca de...

```

Footer.vue M X
< components > ▼ Footer.vue > {} template
1 <template>
2   <footer class="footer-content footer">
3     <ul class="link-aviso">
4       <router-link to="/avisolegal" target="_blank">Aviso Legal</router-link>
5     </ul>
6     <p>{{ new Date().getFullYear() }} XestiónApp — Desenvolvido en Vue 3</p>
7   </footer>
8 </template>
9
10 <script setup></script>
11
12 <style scoped>
13 .footer {
14   background: #1b4965;
15   color: white;
16   text-align: center;
17   padding: 0.2rem 6rem;
18   margin-top: 3rem;
19   font-size: 0.9rem;
20 }
21
22 .link-aviso {
23   display: block;
24   margin-top: 6px;
25   color: #f9d342;
26   cursor: pointer;
27 }
28
29 .footer-content {
30   display: flex;
31   justify-content: center; /* centra horizontalmente */
32   align-items: center; /* alinea verticalmente na mesma liña */
33   gap: 5rem; /* espazo entre o texto e o enlace */
34 }
35
36 </style>

```

O código de NavBar.vue é o seguinte e onde basicamente contén o menú, logo, búsqueda e si fora o caso o acceso de usuarios sería este.

```

1  <template>
2    <nav class="navbar">
3      <div class="logo">XestiónApp</div>
4      <ul class="menu">
5        <li><RouterLink to="/">Inicio</RouterLink></li>
6        <li><RouterLink to="/usuarios">Usuarios</RouterLink></li>
7        <li><RouterLink to="/tarefas">Tarefas</RouterLink></li>
8        <li><RouterLink to="/sobrenos">Sobre nós</RouterLink></li>
9      </ul>
10   </nav>
11 </template>
12
13 <script setup></script>
14
15 <style scoped>
16   .navbar {
17     display: flex;
18     justify-content: space-between;
19     align-items: center;
20     padding: 0.1rem 6rem;
21     background: #1b4965;
22     color: white;
23   }
24   .logo {
25     font-weight: bold;
26     font-size: 1rem;
27   }
28   .menu {
29     display: flex;
30     gap: 0.8rem;
31     list-style: none;
32   }
33   .menu a {
34     color: white;
35     text-decoration: none;
36     font-weight: 500;
37   }
38   .menu a.router-link-active {
39     border-bottom: 0.2rem solid #ffe66d;
40   }
41 </style>

```

En ambos casos aparece a directiva **router-link** que é a que nos permite navegar dentro da páxina vendo diferentes vistas da mesma. Para iso necesitamos un elemento dentro de *Vue.js* chamado, *Vue-Router* que permite facer ditas operacións.

A.2. Configuración vue-router

Vue Router é a librería oficial de **enrutamento** para aplicacións creadas con Vue.js. Permite converter unha aplicación común nunha **SPA (Single Page Application)**, onde a navegación entre "páxinas" non require recargar o navegador.

Con Vue Router podemos:

- Crear **varias vistas** dentro dunha **mesma aplicación**.
- **Definir rutas** como nunha *web* tradicional: */inicio*, */clientes*, */acercadenos...*.
- Navegar entre elas **sen perda de estado**.
- **Traballar con parámetros**, rutas anidadas, protección de rutas, etc.

Nunha SPA real, ao cambiar de páxina **non se carga outra vez o HTML**, só se troca o compoñente que se está mostrando. Vue Router **encárgase de xestionar este proceso**.

Precisa para elo dos seguintes **elementos básicos**:

- **RouterView**: é o compoñente que **di a Vue onde debe renderizar a vista actual**.
- **RouterLink**: é o compoñente equivalente aos enlaces **<a>** nunha web tradicional, **pero non é unha etiqueta HTML**, senón un compoñente de Vue que evita a recarga da páxina.
- **Historial (createWebHistory)**: Vue Router xestiona a navegación usando un modo de historial:
 - **createWeHistory()**: usa a API moderna do navegador (*pushState*), sen "#".
 - **createWebHashHistory()**: usa "#", máis compatible pero menos elegante.
- **Parámetros**: permiten rutas dinámicas (*/usuario/123...*)
- **Navegación programática**: as veces necesitamos abrir ligazóns que non están no menú principal ou ben lanzalo dende un método de JS.

```
import { useRouter } from 'vue-router'
```

```
const router = useRouter()
```

```
router.push('/inicio')
```

O seguinte esquema conceptual preséntanos de forma máis resumida o anterior.

Concepto	Que é	Para que serve
Ruta	Obxecto configuración	Indica que compoñente se mostra nunha URL
RouterLink	Compoñente de navegación	Cambia de páxina sen recargar
RouterView	Zona dinámica	Visualiza o compoñente asociado á ruta actual
Parámetros	Datos na URL	Permiten rutas dinámicas (ex: /usuario/12)
useRoute	Hook	Ler información da ruta (params, query...)
useRouter	Hook	Navegar por código (push, replace...)
Historial	Modo de navegación	Controla como se ve a URL

A utilización de Vue Router, ademais de darlle un aspecto máis profesional, permite unha organización máis limpa do proxecto, traballar con *CRUDs* complexos, autenticación e outras prácticas dende o inicio.

Empecemos pola parte práctica.

Paso 1. Instalación de Vue-Router

```

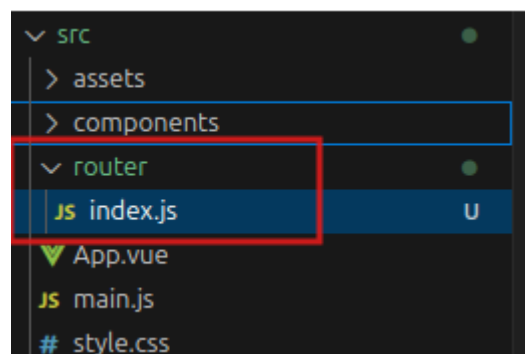
usuario@cursovvue:~/proxectovue$ npm install vue-router
added 2 packages, and audited 38 packages in 4s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

Paso 2. Crear a carpeta **router** e dentro dela o arquivo **index.js**. Pódese chamar de outra maneira pero ese nome é o habitual.



Paso 3. Dentro de ese archivo imos a establecer o contido básico seguinte de acordo o noso menú.

```
index.js U X
> router > JS index.js > default
1 import { createRouter, createWebHistory } from 'vue-router'
2 import Inicio from '../components/Inicio.vue'
3 import XestionUsuarios from '../components/XestionUsuarios.vue'
4 import XestionTarefas from '../components/XestionTarefas.vue'
5 import SobreNos from '../components/SobreNos.vue'
6 import AvisoLegal from '../components/AvisoLegal.vue'
7
8 const routes = [
9   { path: '/', name: 'inicio', component: Inicio },
10  { path: '/usuarios', name: 'usuarios', component: XestionUsuarios },
11  { path: '/tarefas', name: 'tarefas', component: XestionTarefas },
12  { path: '/sobrenos', name: 'sobrenos', component: SobreNos },
13  { path: '/avisolegal', name: 'avisolegal', component: AvisoLegal }
14 ]
15
16 const router = createRouter({
17   history: createWebHistory(),
18   routes
19 })
20
21 export default router
```

Paso 4. Imos a **main.js** para dicirlle a aplicación que ten que usar *router*.

```
JS main.js
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4 import router from './router'
5
6 createApp(App).use(router).mount('#app')
```

Antes de seguir avanzando, nestes momentos vemos que o entorno de programación comeza a darnos un erro, en realidade varios, tantos como compoñentes que declaramos e que aínda non codificamos xa que só temos *XestionUsuarios.vue*.

```

20:43 [vite] Internal server error: Failed to resolve import "../Inicio.vue" from "src/router/index.js". Does the file exist?
plugin: vite:import-analysis
file: /home/usuario/proxectovue/src/router/index.js:2:20
|   import { createRouter, createWebHistory } from 'vue-router'
|   import Inicio from '../Inicio.vue'
|   ^
|   import Usuarios from '../XestionUsuarios.vue'
|   import Tareas from '../XestionTareas.vue'
|   at TransformPluginContext.formatLog (file:///home/usuario/proxectovue/node_modules/vite/dist/node/chunks/config.js:31106:43)
|   at TransformPluginContext.error (file:///home/usuario/proxectovue/node_modules/vite/dist/node/chunks/config.js:31103:14)
|   at normalizeId (file:///home/usuario/proxectovue/node_modules/vite/dist/node/chunks/config.js:29508:18)

```

Por iso, imos crear uns compoñentes básicos, de momento sen ningunha funcionalidade e así corrixir os erros e, ademais, permitiranos comprender como funciona *vue-router*. Neste punto **non nos preocupa o contido das páxinas**, senón comprender **como funciona a navegación**.

É importante que os nomes coincidan en maiúsculas e minúsculas, especialmente en *Linux*.

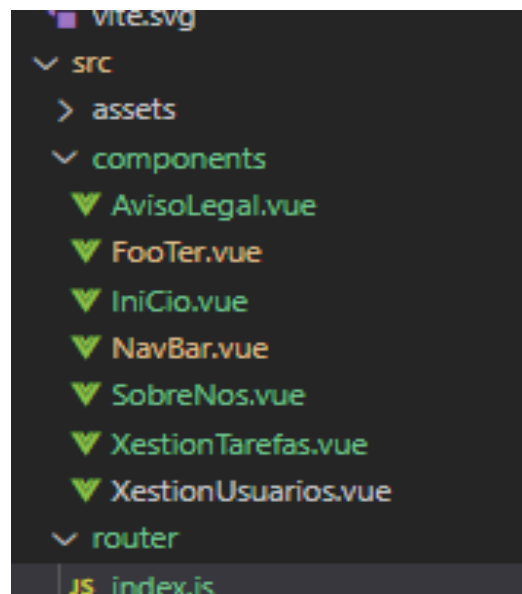
Os código son:

IniCio.vue	XestorTareas.vue	SobreNos.vue	AvisoLegal.vue
<pre> <template> <h2>Inicio</h2> </template> <script setup></script> <style scope></style> </pre>	<pre> <template> <h2>Tareas</h2> </template> <script setup></script> <style scope></style> </pre>	<pre> <template> <h2>SobreNos</h2> </template> <script setup></script> <style scope></style> </pre>	<pre> <template> <h2>Aviso Legal</h2> </template> <script setup></script> <style scope></style> </pre>

Agora paramos o servidor (Ctrl+C) e volvemos a arrincalo e deixa de dar erros.

En realidade, tampouco é estritamente necesario paralo xa que a medida que facemos modificacións no entorno está continuamente probando se todo funciona. Sen embargo, as veces convén facelo.

A imaxe mostra como debería ser a disposición final.



```

usuario@cursovvue:~/proxectovue$ npm run dev

VITE v7.1.12 ready in 408 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

```

Paso 5. Quedamos modificar o **App.vue**. Ata agora, chamaba directamente a **XestionUsuarios.vue**, pero non é que queremos, o que buscamos e que cargue a páxina inicial "/" e que poña "a funcionar" o **enrutamento**, cargando a función **router-view**. **Para mellorar o deseño** fai os cambios que se indican na zona **style** o ben podes tomar o arquivo **App.vue** do almacén da Unidade.

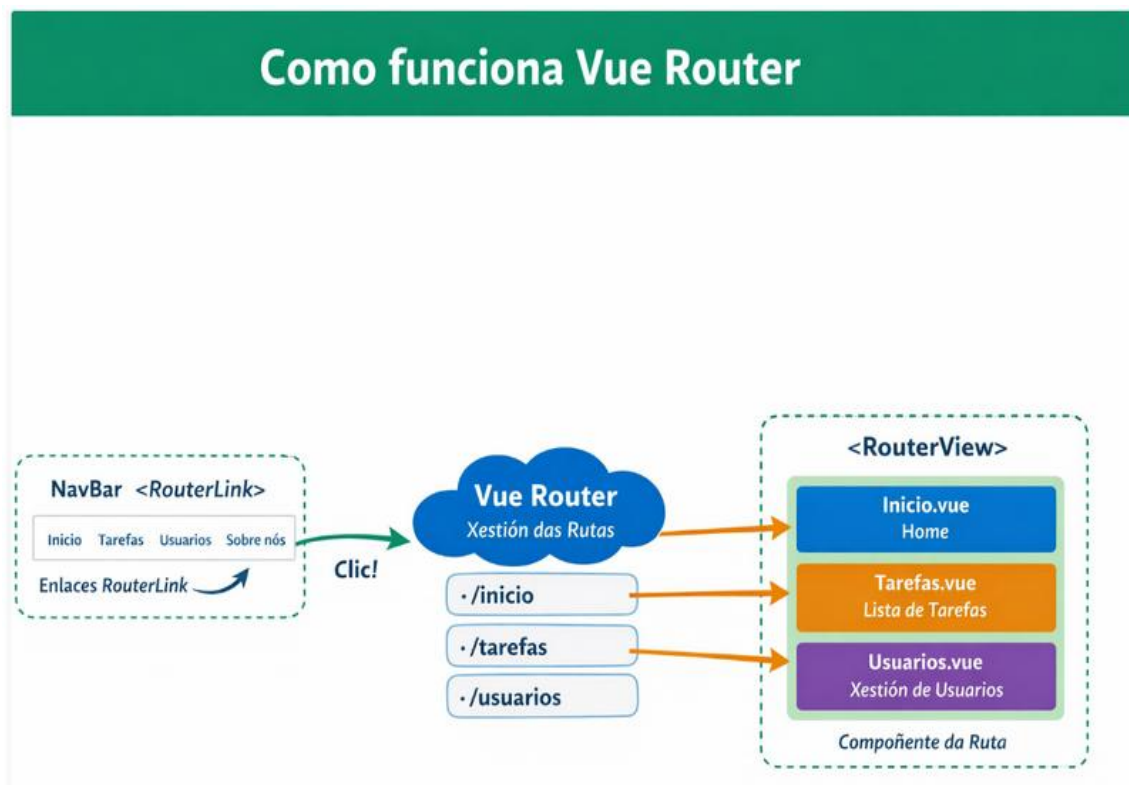
```

App.vue M X
c> ▼ App.vue > {} style
1  <template>
2    <div id="app">
3      <NavBar />
4      <!-- Cargamos o xestor de rutas-->
5      <main >
6        <router-view />
7      </main>
8      <Footer />
9    </div>
10 </template>
11
12 <script setup>
13 import NavBar from '../components/NavBar.vue'
14 import Footer from '../components/Footer.vue'
15 </script>
16
17 <style>
18 body {
19   margin: 0;
20   background: #f6f6f6;
21 }
22
23 #app { width: 80%; margin: 0 auto;}
24
25 .navbar,
26 .footer {
27   position: fixed;
28   left: 50%;
29   transform: translateX(-50%);
30   width: 80%;
31   z-index: 1000;
32 }
33
34 .navbar {top: 0; }
35
36 .footer {bottom: 0; }
37
38 main { margin-top: 30px; margin-bottom: 30px; }
39 </style>

```

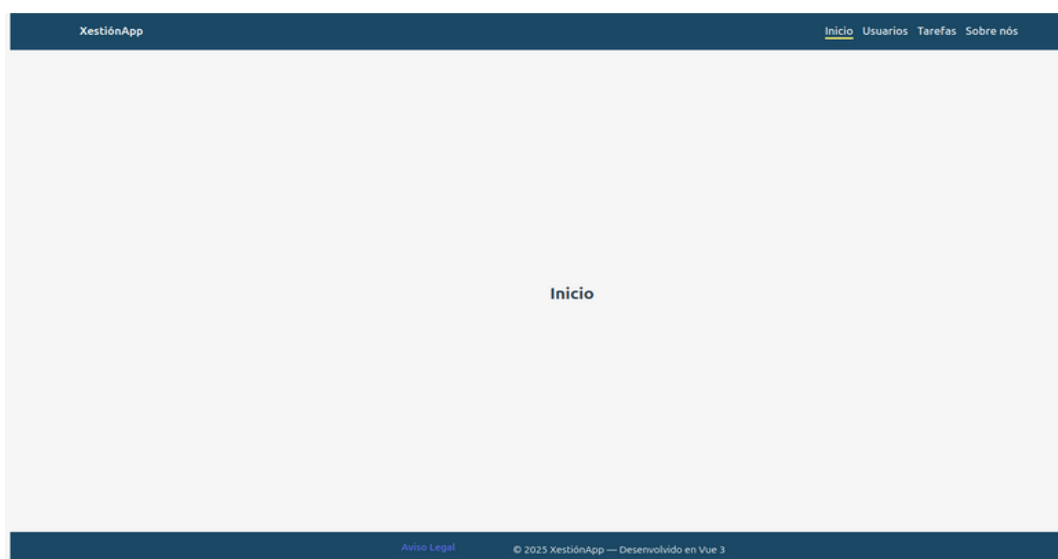
Unha vez feito, xa podemos navegar polos diferentes ligazóns da aplicación.

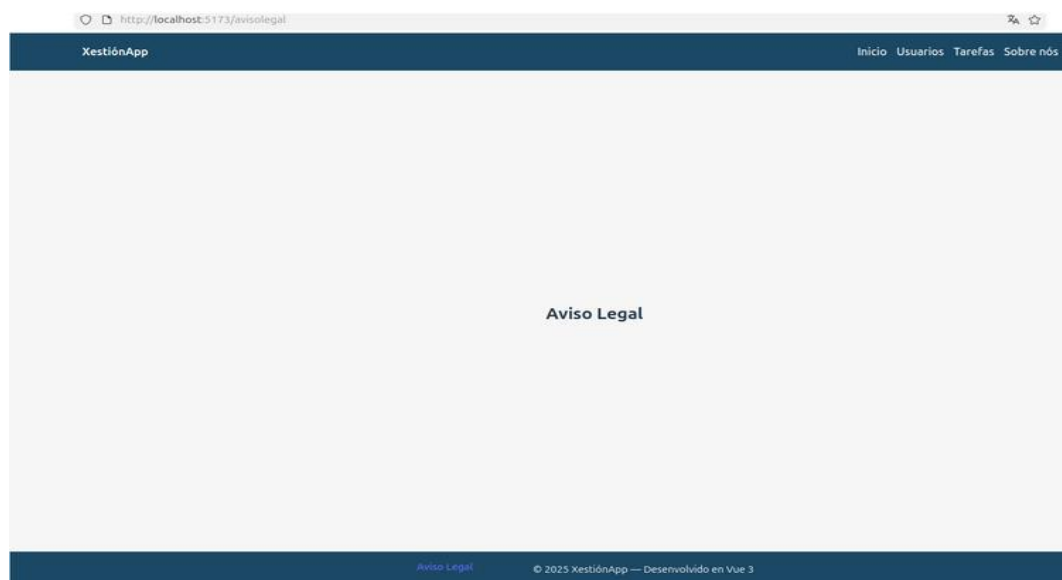
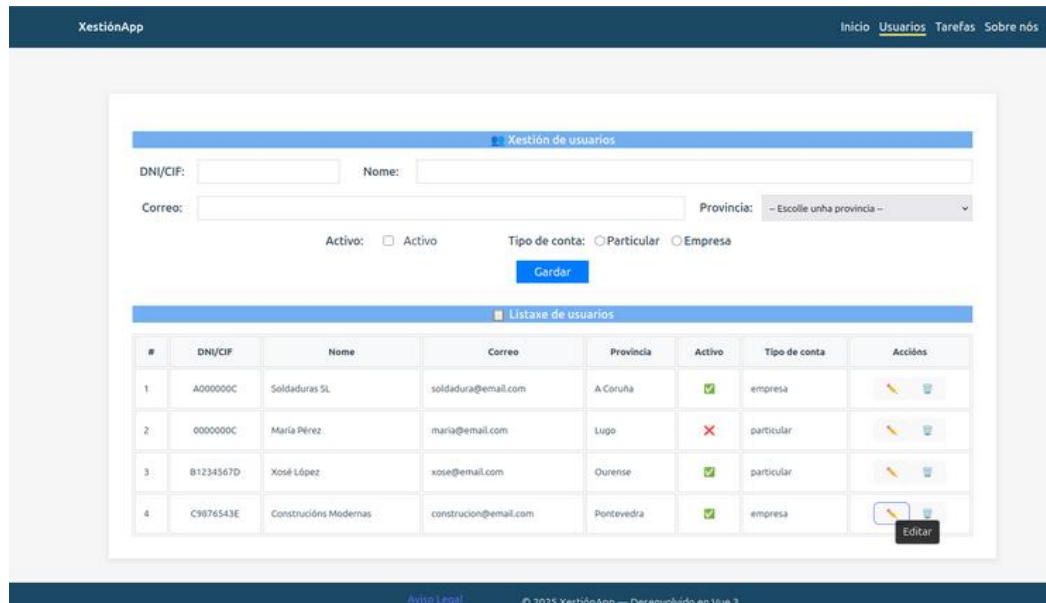
Nas seguinte imaxes móstranos como funciona Vue Router.



Basicamente, cada vez que pulsamos un ligazón (*router-link*), a librería *Vue Router* busca en ***index.js*** a que compoñente da ruta se corresponde para que *Vue* poda chamalo e cargalo.

As páxinas vanse cargando segundo as vamos chamando.





En resumo, podemos pensar en **<router-view>** como unha “ventá” que cambia de contido segundo a ruta activa.

A.3. Comunicación entre compoñentes

A **comunicación entre compoñentes en Vue** refírese ao intercambio de datos e eventos entre distintos compoñentes dunha aplicación, permitindo que un compoñente "*pai*" pase información aos seus "*fillos*" mediante **props**, e que estes envíen de volta eventos usando **\$emit**.

- **Compoñente Pai:** É o que ten o control dos datos principais. Exemplo: *XestionUsuarios.vue* que contén a lista de usuarios.
- **Compoñente Fillo:** Recibe datos do pai e pode facer operacións sobre eses datos ou amosalos. Exemplo: *XestionTarefas.vue* que amosa as tarefas dun usuario concreto.

O *fillo* **non modifica directamente** os datos do *pai*, só recibe información ou envía eventos para que o pai actúe.

As **ferramentas** que usamos en Vue **para a comunicación** son:

Concepto – Directiva	Función
props	Permite ao pai pasar datos ao fillo. <i>Exemplo:</i> <code><Fillo :usuario="usuarioSeleccionado"/></code>
\$emit	Permite ao fillo enviar eventos ao pai. <i>Exemplo:</i> <code>emit('tarefaActualizada', novaTarefa)</code>
v-model	Pode usarse en compoñentes fillos para crear unha comunicación bidireccional simplificada (vincula prop + evento)
computed	Útil no pai para calcular datos derivados. <i>Exemplo, contar tarefas dun usuario</i>
v-for	No fillo, para <i>renderizar</i> listas de datos que recibe do pai
v-if	Mostrar elementos no fillo só se hai datos dispoñibles

Imos a entrar en materia.

No **almacén da Unidade 3** temos o compoñente *XestiónTarefas.vue* co código que nos permite mostrar as tarefas dun usuario. De momento, son todas iguais. Cando empecemos na UD4 entón poderemos acercarnos a un suposto máis real.

Empezamos **modificando o compoñente pai ou XestiónUsuarios.vue**. A idea é que cando premamos e carguemos os datos un usuario no formulario, o compoñente fillo *XestionTarefas.vue* cargue as súas tarefas.

- **Zona script.** Engadimos dúas cousas
 - unha referencia para o usuario seleccionado e as súas tarefas no pai
 - aproveitando o botón do lapis ou editar cargar os datos de usuario en *usuario seleccionado* e as *tarefas*

```

/// referencias ao usuario seleccionado e as súas tarefas

const usuarioSeleccionado = ref(null)
const tarefasUsuario = ref([])

/// Zona de métodos ou funcións

function gardarUsuario() {
  usuarios.value.push({ ...novoUsuario }) //engade o novo usuario á lista (copia do obxecto)
  Object.assign(novoUsuario, { nome: "", correo: "", idade: null, provincia: "", activo: false, se
}

function eliminarUsuario(index) {
  usuarios.value.splice(index, 1); //elimina o usuario da lista
}

function editarUsuario(index) {
  const usuario = usuarios.value[index]; //carga os datos do usuario elixido no formulario
  Object.assign(novoUsuario, usuario); // carga os datos do usuario no formulario recorda v-model
  // Actualizamos a comunicación co fillo

  usuarioSeleccionado.value = usuario
  tarefasUsuario.value = [
    // Aquí poñemos tarefas de exemplo ou as que veñan da base de datos
    { titulo: "Tarefa 1", dataLimite: "2025-12-31", completada: false },
    { titulo: "Tarefa 2", dataLimite: "2026-01-15", completada: true }
  ]
}

</script>

```

- **Zona Template:** engadimos a referencia a *XestionTarefas.vue*. Isto obríganos a que na zona de script, teñamos que importar o compoñente fillo.

```

</div>
<!-- Aquí engadimos o fillo -->
<XestionTarefas
  :usuario="usuarioSeleccionado"
  :tarefas="tarefasUsuario"
/>
</template>

<script setup>
  /// Zona de declaracións
  import { ref, reactive, onMounted } from 'vue'
  // Importamos o compoñente fillo
  import XestionTarefas from './XestionTarefas.vue'

  const usuarios = ref([]) //almacena la lista de usuarios e os seus cambios

  const novoUsuario = reactive({
    dni: "",
    nome: "",
    correo: "",
    provincia: "",
    activo: false,
    tipoCuenta: ""
  })

```

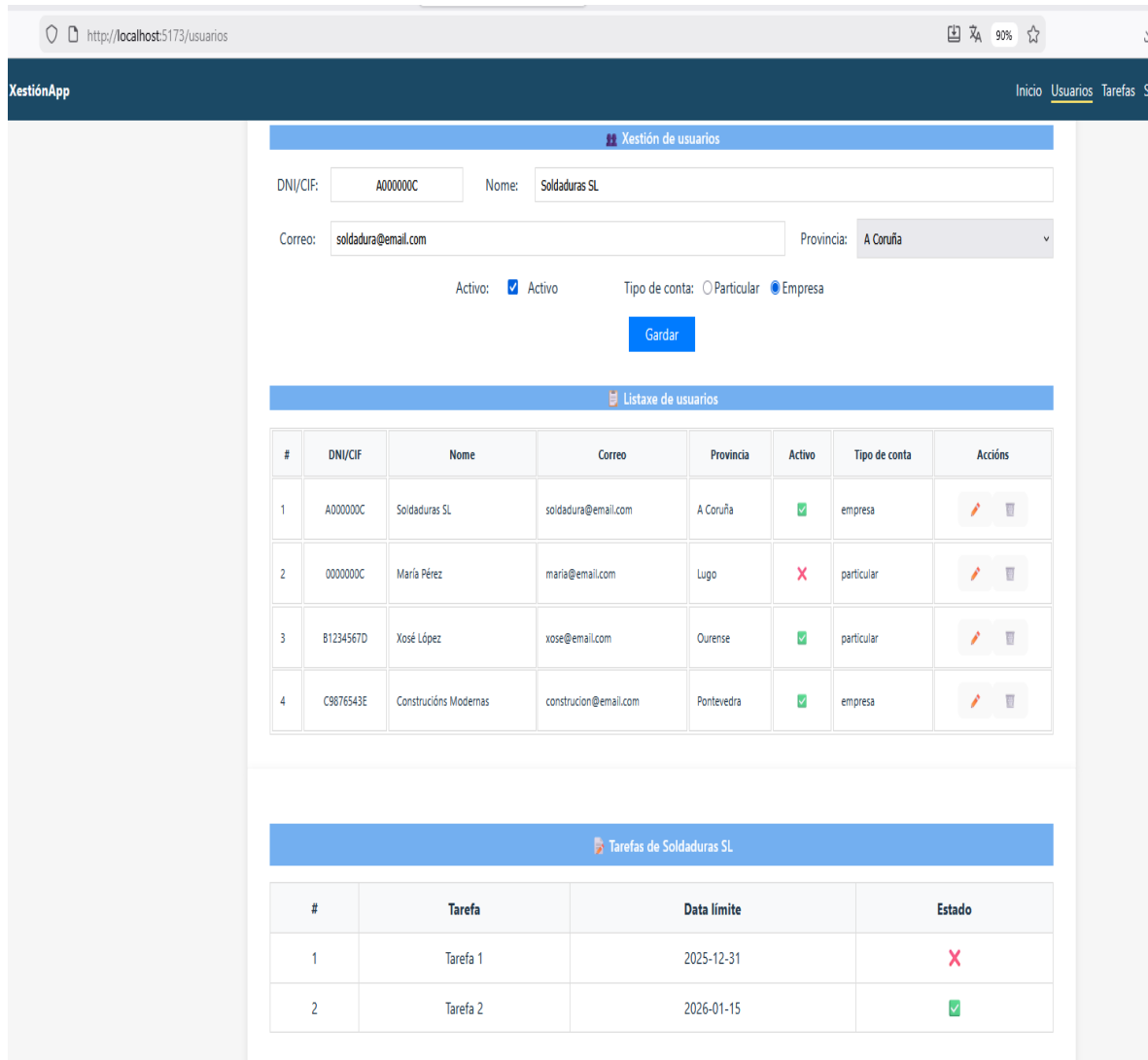
Estas últimas liñas é o que conecta o **compoñente pai** (*XestionUsuarios.vue*) co **fillo** (*XestionTarefas.vue*) mediante **props**, que son como “canles de comunicación de datos unidireccionais”.

- **:usuario="usuarioSeleccionado"**: está a pasar un obxecto `usuarioSeleccionado` do pai ao fillo. O “:” é un **shorthand (expresión curta)** de **v-bind**, que indica que o valor é unha expresión de *JavaScript* (non un *string* literal). No fillo, podemos acceder a ese obxecto directamente co nome **usuario**.
- **:tarefas="tarefasDoUsuario"**: de igual forma, pasa ao fillo a lista de tarefas correspondente ao usuario seleccionado. No fillo, está dispoñible como `tarefas`.

Porque é importante:

1. Permite que o fillo **se mantenga reactivo**: cada vez que o pai cambia `usuarioSeleccionado` ou `tarefasDoUsuario`, o fillo actualízase automaticamente sen que o fillo teña que facer nada.
2. Mantén unha separación de responsabilidades: o pai controla os datos e o fillo só os amosa.
3. Facilita o fluxo de comunicación **unidireccional**, que é a forma recomendada en Vue para evitar problemas de sincronización ou efectos colaterais.

Aínda que non é unha limitación técnica, senón unha consecuencia do fluxo de datos entre compoñentes pai → fillo, o uso de **props** implica que **temos que renderizar no mesmo compoñente, neste caso o pai, o compoñente fillo, neste caso as tarefas** o que as veces non é algo visualmente atractivo, como se pode ver na imaxe.



Ata agora vimos comunicación **de pai a fillo**. Agora imos ver que pasa cando o fillo necesita **avisar ao pai** de que ocorreu algo usando **\$emit**.

O compoñente `XestiónUsuarios.vue` será o pai, pero tamén recibirá sinais do fillo mediante a sinal **\$emit**.

Faremos algo sinxelo e realista:

- **Marcar unha tarefa** como completada desde `XestionTareas.vue`
- O fillo emite: *"esta tarefa cambiou"*
- O pai decide que facer con ese cambio

Para iso en *XestiónTarefas.vue* só engadimos o seguinte que permite que o compoñente fillo envíe unha sinal ao compoñente pai o *XestiónUsuarios.vu*.

```

1 <template>
2   <div class="xestion-tarefas">
10     <table v-else>
11       <thead>
18     </thead>
19     <tbody>
20       <tr v-for="(t, index) in tarefas" :key="index">
21         <td style="text-align: center;">{{ index + 1 }}</td>
22         <td style="text-align: center;">{{ t.titulo }}</td>
23         <td style="text-align: center;">{{ t.dataLimite }}</td>
24         <td style="text-align: center;">{{ t.completada ? '✅' : '❌' }}</td>
25         <!-- Botón para completar tarefa -->
26         <td style="text-align: center;">
27           <button
28             @click="completarTarefa(index)"
29             :disabled="t.completada"
30           >
31             ✓
32           </button>
33         </td>
34       </tr>
35     </tbody>
36   </table>
37 </div>
38 </template>
39
40 <script setup>
41 const props = defineProps({
42   usuario: { type: Object, default: null },
43   tarefas: { type: Array, default: () => [] }
44 })
45
46 /// Evento para comunicar co compoñente pai o XestiónUsuarios
47 const emit = defineEmits(['tarefa-completada'])
48
49 /// Método que indica que unha tarefa foi completada
50 function completarTarefa(index) {
51   emit('tarefa-completada', index)
52 }
53 </script>
54

```

Mentres, no compoñente *XestiónUsuarios.vue* engadimos a función marcar a tarefa completada.

```

XestionTarefas.vue X XestionUsuarios.vue M X App.vue M
c > components > XestionUsuarios.vue > {} template
1 <template>
2 <div class="xestion-usuarios">
3 <p v-else>Non hai usuarios cargados.</p>
89 </div>
90 <!-- Aquí engadimos o fillo -->
91 <XestionTarefas
92 :usuario="usuarioSeleccionado"
93 :tarefas="tarefasUsuario"
94 @tarefa-completada="marcarTarefaCompletada"
95 />
96
97 </template>
98
99 <script setup>
100 /// Zona de declaracións
101 import { ref, reactive, onMounted } from 'vue'
102 // Importamos o compoñente fillo
103 import XestionTarefas from './XestionTarefas.vue'
104
105 const usuarios = ref([]) //almacena la lista de usuarios e os seus cambios
106
107 const novoUsuario = reactive({
108   dni: "",
109   nome: "",
110   correo: "",
111   provincia: "",
112   activo: false,
113   tipoCuenta: ""
114 })
115
116 /// Zona de ciclo de vida
117
118 onMounted(() => { //sempre se cargan estos usuarios de exemplo ao iniciar o componente
119   usuarios.value = [
120     { dni: "A000000C", nome: "Soldaduras SL", correo: "soldadura@email.com", provincia: "A Coruña", activo: true, tipoCuenta: "A", },
121     { dni: "0000000C", nome: "María Pérez", correo: "maria@email.com", provincia: "Lugo", activo: false, tipoCuenta: "A", },
122     { dni: "B1234567D", nome: "Xosé López", correo: "xose@email.com", provincia: "Ourense", activo: true, tipoCuenta: "A", },
123     { dni: "C9876543E", nome: "Construcións Modernas", correo: "construcion@email.com", provincia: "Pontevedra", activo: true, tipoCuenta: "A", }
124   ]
125 })
126
127 /// referencias ao usuario seleccionado e as súas tarefas
128
129 const usuarioSeleccionado = ref(null)
130 const tarefasUsuario = ref([])
131
132 /// Método para marcar unha tarefa como completada
133 function marcarTarefaCompletada(index) {
134   tarefasUsuario.value[index].completada = true
135 }
136
137 /// Zona de métodos ou funcións

```

XestionUsuarios.vue é o **pai**, porque ten os datos reais. *XestionTarefas.vue* é o **fillo**, porque só amosa e interactúa. O fillo **non cambia datos directamente**. Cando pasa algo importante usa **\$emit**, e o pai decide como actualizar o estado da tarefa.

Nas imaxes posteriores vemos o funcionamento.

DNI/CIF: Nome:

Correo: Provincia:

Activo: ☒ Activo Tipo de conta: ☐ Particular ☒ Empresa

Listaxe de usuarios

#	DNI/CIF	Nome	Correo	Provincia	Activo	Tipo de conta	Accións
1	A000000C	Soldaduras SL	soldadura@email.com	A Coruña	☒	empresa	✎ ✖
2	0000000C	María Pérez	maria@email.com	Lugo	☐	particular	✎ ✖
3	B1234567D	Xosé López	xose@email.com	Ourense	☒	particular	✎ ✖
4	C9876543E	Construcións Modernas	construcion@email.com	Pontevedra	☒	empresa	✎ ✖

Tarefas de Soldaduras SL

#	Tarefa	Data límite	Estado	Accións
1	Tarefa 1	2025-12-31	☐	✓
2	Tarefa 2	2026-01-15	☒	✓

Se pulsamos en Accións na Tarefa 1, un evento dende o compoñente fillo rexístrase no compoñente pai.

http://localhost:5173/usuarios

Inicio U

DNI/CIF: Nome:

Correo: Provincia:

Activo: ☒ Activo Tipo de conta: ☐ Particular ☒ Empresa

Listaxe de usuarios

#	DNI/CIF	Nome	Correo	Provincia	Activo	Tipo de conta	Accións
1	A000000C	Soldaduras SL	soldadura@email.com	A Coruña	☒	empresa	✎ ✖
2	0000000C	María Pérez	maria@email.com	Lugo	☐	particular	✎ ✖
3	B1234567D	Xosé López	xose@email.com	Ourense	☒	particular	✎ ✖
4	C9876543E	Construcións Modernas	construcion@email.com	Pontevedra	☒	empresa	✎ ✖

Tarefas de Soldaduras SL

#	Tarefa	Data límite	Estado	Accións
1	Tarefa 1	2025-12-31	☒	✓
2	Tarefa 2	2026-01-15	☒	✓

A.4. Resumo

- **Enrutamento en Vue:** Comprender e aplicar rutas para navegar entre compoñentes dentro dunha SPA.
- **Rutas comodín:** Xestionar *URLs* inexistentes mostrando un compoñente de "Páxina non encontrada".
- **Comunicación pai → fillo:** Pasar datos mediante ***props*** para amosar información adicional nos compoñentes fillos.
- **Estado básico compartido:** Mantemento de datos seleccionados e actualización de compoñentes relacionados sen ***backend***.